

SOFTWARE ENGINEERING – LEVEL 3 · MINI-PROJECT

Software Requirements Specification (SRS)

Electronics Store Inventory Management System (ESIMS) · Version 1.0

Project Title	Electronics Store Inventory Management System (ESIMS)
Project ID	ESIMS-2026-Go7
Group Number	7
Team Members	C. Harshal Rajesh – PES1UG24CS118 Ayaan Ahmed – PES1UG24CS102
Course	UE24CS341A – Software Engineering
Semester	V Semester
Development Methodology	Agile
Document Version	1.0
Submission Format	PDF

Table of Contents

1.	Introduction
2.	Overall Description
3.	External Interface Requirements
4.	Analysis Models
5.	System Features — Functional Requirements
6.	Non-Functional Requirements
7.	Security Requirements
8.	Business / Domain Rules
9.	Other Requirements — Database / Field Layout
A.	Appendix A — Glossary
B.	Appendix B — Requirement Traceability Matrix

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the **Electronics Store Inventory Management System (ESIMS)**, Version 1.0. ESIMS is a web-based application that enables an electronics store to manage its component catalogue, record stock-in and stock-out movements, monitor stock availability, and generate inventory reports and dashboards. This document describes the intended behaviour, scope, interfaces, constraints, and requirements against which the system will be designed, implemented, tested, and validated.

1.2 Document Conventions

- Every requirement is uniquely identified: FR-xxx (functional), NFR-xxx (non-functional), SEC-xxx (security) and UC-xx (use case).
- The keyword “**shall**” denotes a mandatory requirement.
- Priority is expressed as High, Medium or Low; each requirement lists a planned verification method.
- The document structure is adapted from IEEE Std 830-1998 and the PES University SE Level 3 submission guide.

1.3 Intended Audience and Reading Suggestions

This document is intended for the course faculty and evaluators, the project team members, and any future maintainers of the system. Readers seeking an overview should read Sections 1–2; those interested in interfaces and models should read Sections 3–4; the detailed requirements are in Sections 5–8; and supporting detail (field layout, glossary, traceability) is in Section 9 and the appendices.

Intended Users of the System

- **Administrator** — a trained, higher-privilege user who manages users, catalogue data and reports.
- **Staff** — a non-technical operational user who maintains components and records stock movements.
- **System** — automated, scheduled logic that derives availability status and raises low-stock alerts.

1.4 Product Scope

ESIMS v1.0 is a single-location (single-tenant) inventory system focused on component catalogue management, stock movement tracking, availability classification, reporting and an operational dashboard. The scope of the first release is summarised below.

In Scope	Out of Scope
Authentication & role-based access (Admin, Staff)	POS / billing / customer checkout
Component CRUD, search and filtering	Online payment / bank settlement
Category & supplier management	Barcode / RFID hardware integration
Stock-in / stock-out with validation	Purchase-order / procurement automation
Availability classification & low-stock alerts	Accounting / ERP / GST integration
Availability report + CSV/PDF export	Multi-branch / multi-warehouse deployment
Dashboard with KPIs and charts	AI demand forecasting
Audit log	Native mobile apps

1.5 References

- Jackfruit – Software Engineering Project Guidelines 2026.
- PES University SE Level 3 Student Submission Guide (Coffee Kiosk exemplar).
- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications (structure adapted).

2. Overall Description

2.1 Product Perspective

ESIMS is a brand-new, standalone web application — not an add-on to any existing product. Internally it is organised in clear layers: the web UI talks to an API / controller layer, which calls the business logic (services), which reaches the relational database through a data-access layer, with helper utilities handling exports and notifications.

2.2 Product Functions

- Authenticate users and enforce roles.
- Maintain the component catalogue (CRUD + search / filter).
- Maintain categories and suppliers.
- Record stock-in / stock-out and keep quantities accurate.
- Classify availability and raise low-stock alerts.
- Generate and export availability reports.
- Present an inventory dashboard.
- Maintain an audit log.

2.3 User Classes and Characteristics

User class	Privileges	Characteristics
Administrator	Full access: user management; delete components/categories/suppliers; all reports and dashboard	Trained, higher privilege
Staff	Add/edit components; record stock movements; view reports	Non-technical, frequent operational use
System (scheduled/derived)	Recompute availability status; raise low-stock alerts	Automated, no UI

2.4 Operating Environment

- **Client:** modern web browser (Chrome, Edge, Firefox) on desktop / laptop.
- **Server:** any OS running Python / Django, served via WSGI / ASGI.
- **Database:** PostgreSQL 14+ (SQLite for local development).
- **Network:** HTTP / HTTPS over LAN or internet.

2.5 Design and Implementation Constraints

- Web application built with Python / Django + Django REST Framework (approved stack; MERN or another stack may be substituted with faculty approval).
- Relational database with referential integrity and transactions.
- Passwords stored only as salted hashes.
- HTTPS in any deployed environment.
- Source managed on GitHub with an automated CI pipeline.

2.6 Assumptions and Dependencies

- Users have valid accounts created by an administrator.
- The store operates a single location / warehouse (single-tenant) for v1.0.
- A stable browser and network connection are available.
- No real payment / POS integration is required for v1.0.

3. External Interface Requirements

3.1 User Interfaces

- Login screen.
- Inventory list screen with search / filter and availability badges.
- Add / Edit component form with inline field validation.
- Stock-in / stock-out transaction form.
- Category & supplier management screens.
- Availability report screen with filters and an export button.
- Dashboard with KPI cards and charts.
- Consistent, responsive Bootstrap layout; validation and error messages shown inline.

3.2 Hardware Interfaces

Standard desktop / laptop hardware only; no specialized devices. (A barcode scanner is a possible future enhancement and is out of scope for v1.0.)

3.3 Software Interfaces

- REST API (JSON) between the frontend and backend.
- Django ORM to PostgreSQL.
- CSV / PDF export library for reports.
- Optional email / notification service for low-stock alerts (mocked in v1.0).

3.4 Communication Interfaces

- HTTPS for all client–server communication.
- Standard HTTP status codes and JSON request / response payloads for the API.

4. Analysis Models

4.1 Use Case Diagram

Actors: Administrator, Staff, System. **Use cases:** Login, Manage Components, Search Inventory, Manage Categories / Suppliers, Record Stock Movement, Generate Availability Report, View Dashboard, Manage Users.

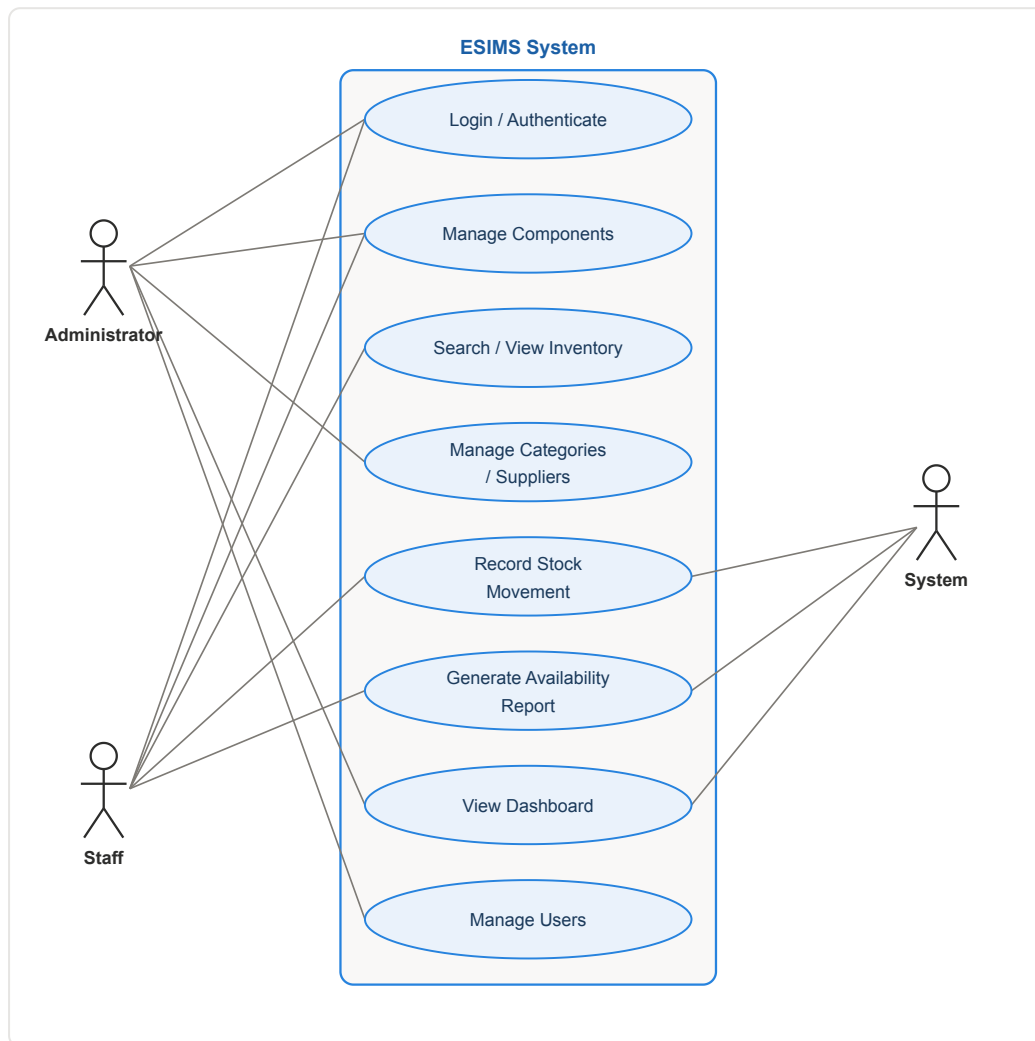


Figure 1 — ESIMS Use Case Diagram

4.2 Use Case Descriptions

UC ID	Use Case	Primary actor	Precondition	Outcome
UC-01	Login / Authenticate	Admin / Staff	Account exists	Authenticated session with role
UC-02	Add Component	Staff / Admin	Logged in	Component created with unique SKU
UC-03	Edit Component	Staff / Admin	Component exists	Component details updated

UC ID	Use Case	Primary actor	Precondition	Outcome
UC-04	Delete Component	Admin	Component exists	Component soft-deleted
UC-05	Search / View Inventory	Staff / Admin	Logged in	Filtered component list displayed
UC-06	Record Stock-in / Stock-out	Staff / Admin	Component exists	Quantity updated, transaction logged
UC-07	Manage Categories / Suppliers	Admin	Logged in	Category / supplier created / updated / removed
UC-08	Generate Availability Report	Staff / Admin	Logged in	Report displayed and exportable
UC-09	View Dashboard	Admin	Logged in	KPIs and charts displayed
UC-10	Manage Users	Admin	Admin logged in	Staff account created / disabled

4.3 Domain / ER Model

Entities: User, Category, Supplier, Component, StockTransaction, AuditLog.

Key relationships:

- Category 1 — * Component
- Supplier 1 — * Component
- Component 1 — * StockTransaction
- User 1 — * StockTransaction (performed_by)
- User 1 — * AuditLog

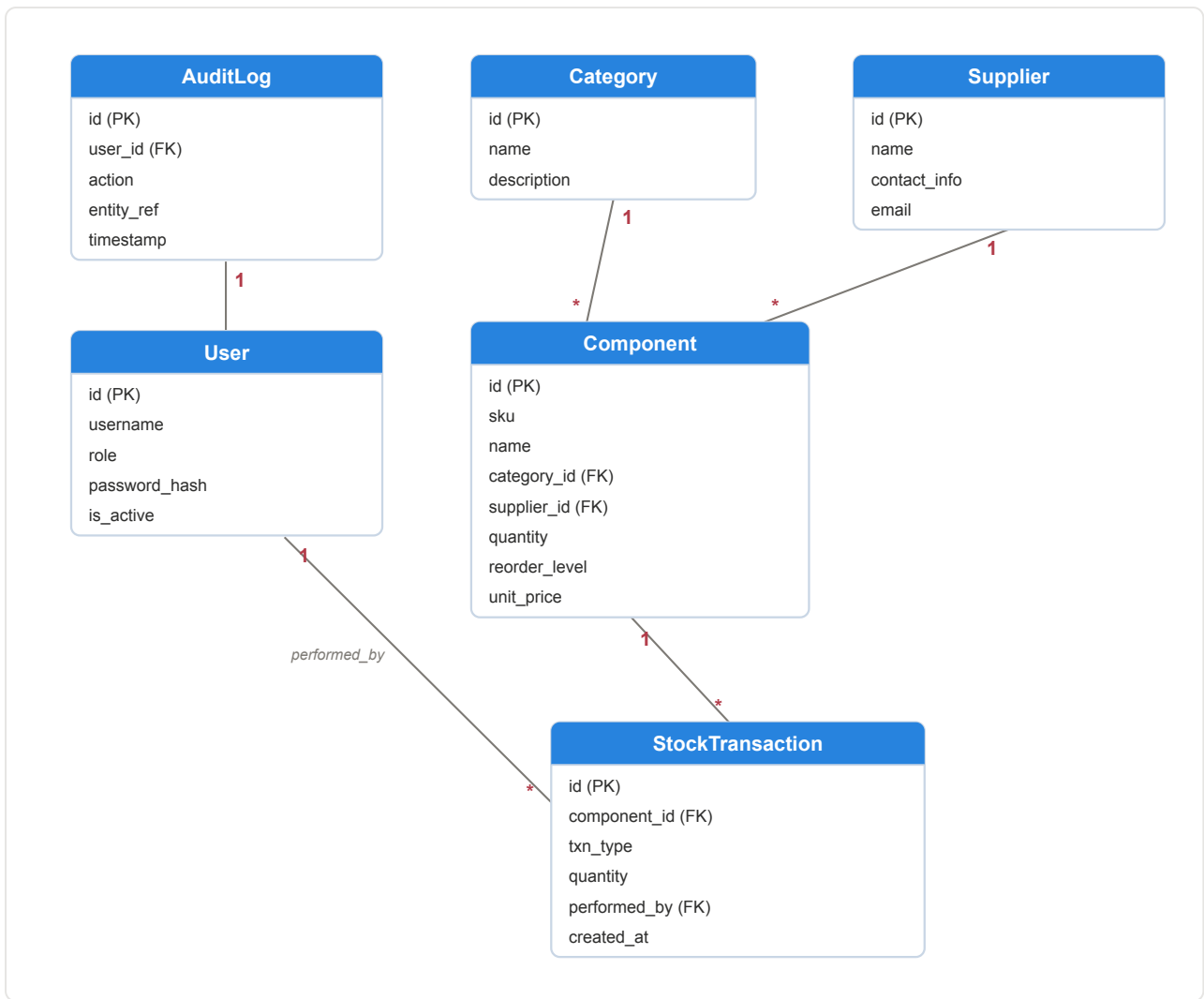


Figure 2 — ESIMS Entity-Relationship Diagram (1 — * denotes one-to-many)

5. System Features — Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow an administrator to create a user account with a unique username / email.	High	Functional test
FR-002	The system shall authenticate a user by username / email and password before granting access.	High	Functional test
FR-003	The system shall enforce role-based access so that Staff cannot perform Administrator-only actions.	High	Authorization test
FR-004	The system shall allow an authenticated user to add a component with name, SKU, category, supplier, unit price, quantity and reorder level.	High	Functional test
FR-005	The system shall reject a component whose SKU / code is not unique.	High	Negative test
FR-006	The system shall allow a user to edit the details of an existing component.	High	Functional test
FR-007	The system shall allow an administrator to delete (soft-delete) a component.	Medium	Functional test
FR-008	The system shall display a paginated, searchable list of components with current quantity and availability status.	High	System test
FR-009	The system shall allow filtering of components by category, supplier and availability status.	High	System test
FR-010	The system shall allow a user to record a stock-in transaction that increases a component's quantity.	High	Integration test
FR-011	The system shall allow a user to record a stock-out transaction and shall prevent a component's quantity from going below zero.	High	Negative + positive test
FR-012	The system shall automatically set availability to “Low Stock” when quantity $\leq$ reorder level and “Out of Stock” when quantity = 0.	High	Unit + system test
FR-013	The system shall allow an administrator to add, edit and delete categories.	Medium	Functional test
FR-014	The system shall allow an administrator to add, edit and delete suppliers.	Medium	Functional test
FR-015	The system shall generate an availability report of components, filterable by category, supplier and status.	High	System test
FR-016	The system shall allow the availability report to be exported as CSV and / or PDF.	Medium	Functional test
FR-017	The system shall display a dashboard showing total components, total stock value, low-stock count and out-of-stock count, with at least one chart.	High	System test
FR-018	The system shall record every stock transaction with component, type, quantity, user and timestamp.	High	Integration test

ID	Requirement	Priority	Verification
FR-019	The system shall log security-relevant events and deletions in an audit log.	Medium	System test

6. Non-Functional Requirements

ID	Category	Measurable requirement	Verification
NFR-001	Performance	95% of inventory list / search requests shall respond within 2 seconds for a catalogue of up to 5,000 components under the defined test load.	Performance test
NFR-002	Availability	The application shall be available at least 99% during store operating hours, excluding planned maintenance.	Operational review
NFR-003	Usability	A trained staff member shall be able to add a new component in no more than 3 primary screens.	Usability walkthrough
NFR-004	Reliability	Concurrent stock-out transactions shall never reduce a component's quantity below zero.	Concurrency / integration test
NFR-005	Maintainability	Business logic shall be organized into identifiable modules / services with automated unit tests.	Code review
NFR-006	Scalability	The system shall support at least 5,000 components and 10 concurrent users without functional degradation.	Load test
NFR-007	Portability	The web UI shall function on the latest versions of Chrome, Edge and Firefox.	Cross-browser test
NFR-008	Testability	Core services shall expose testable interfaces independent of the UI.	Unit / integration test

7. Security Requirements

ID	Security requirement	Validation approach
SEC-001	Passwords shall be stored only as salted hashes, never as plaintext.	Code / configuration review
SEC-002	Role-based access shall prevent Staff from accessing Administrator functions (delete, user management).	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied input to prevent SQL injection and XSS.	Negative / security tests
SEC-004	Authenticated sessions / tokens shall expire, and invalid or expired credentials shall be rejected.	Security test
SEC-005	State-changing requests shall be protected against CSRF.	Security test
SEC-006	Security-relevant events (failed login, unauthorized access, deletions) shall be logged.	System test

8. Business / Domain Rules

- A component's quantity shall never be negative.
- Every component shall have a unique SKU / code and belong to exactly one category.
- Reorder level shall be a non-negative integer.
- A stock-out shall be allowed only if sufficient quantity is available.
- Only an Administrator may delete components, categories, suppliers or user accounts.
- A category or supplier referenced by at least one component shall not be hard-deleted.
- Availability status is **derived, not manually set**: Out of Stock (qty = 0), Low Stock (0 < qty ≤ reorder level), In Stock (qty > reorder level).

9. Other Requirements — Database / Field Layout

Field	Length	Data type	Description	Mandatory
user_id	36	UUID / String	Unique user identifier	Y
username	50	String	Unique login name	Y
password_hash	128	String	Salted password hash	Y
role	20	Enum (Admin / Staff)	Access role	Y
category_id	36	UUID / String	Unique category identifier	Y
category_name	60	String	Category name (unique)	Y
supplier_id	36	UUID / String	Unique supplier identifier	Y
supplier_name	100	String	Supplier name	Y
component_id	36	UUID / String	Unique component identifier	Y
sku	40	String	Unique component code	Y
component_name	100	String	Display name	Y
unit_price	10,2	Decimal	Unit price	Y
quantity	10	Integer	Current stock quantity	Y
reorder_level	10	Integer	Low-stock threshold	Y
availability_status	15	Enum	In / Low / Out of stock (derived)	Y
txn_id	36	UUID / String	Stock transaction identifier	Y
txn_type	10	Enum (IN / OUT)	Transaction type	Y
txn_quantity	10	Integer	Quantity moved	Y
txn_timestamp	—	DateTime	When the transaction occurred	Y

Appendix A — Glossary

Term	Meaning
SKU	Stock Keeping Unit — unique code identifying a component
Reorder level	Quantity threshold at / below which a component is considered low on stock
Stock-in	Transaction that increases quantity (restock / receipt)
Stock-out	Transaction that decreases quantity (issue / sale / usage)
RBAC	Role-Based Access Control
KPI	Key Performance Indicator
CRUD	Create, Read, Update, Delete
Availability status	Derived state: In Stock / Low Stock / Out of Stock

Appendix B — Requirement Traceability Matrix (initial)

Req ID	Description	Use Case	Module / Owner	Planned Test Case
FR-002	Authenticate user	UC-01	Auth (Ayaan)	UT-AUTH-01 / ST-AUTH-01
FR-004	Add component	UC-02	Inventory (Ayaan)	UT-INV-01 / ST-INV-01
FR-008	List / search inventory	UC-05	Inventory (Ayaan)	ST-INV-02
FR-011	Stock-out with validation	UC-06	Stock (Harshal)	UT-STK-01 / IT-STK-01
FR-012	Availability classification	UC-06 / UC-08	Stock (Harshal)	UT-STK-02
FR-015	Availability report	UC-08	Reporting (Harshal)	ST-RPT-01
FR-017	Dashboard KPIs	UC-09	Dashboard (Harshal)	ST-DSH-01
SEC-002	RBAC enforcement	UC-01	Auth (Ayaan)	ST-SEC-01
NFR-001	Performance	—	API layer	PERF-01

The RTM will be extended with architecture references, code-file references and executed test results during the Design and Implementation & Testing phases.